Міністерство освіти і науки України
Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки
Кафедра обчислювальної техніки

Лабораторна робота №2

з дисципліни
«Об’єктно-орієнтоване програмування»
на тему
«Розробка графічного редактора об’єктів»

Виконала:

Студентка групи ІМ-55

Долгіх Дар’я Михайлівна
Номер у списку групи: 7
Перевірив:
Рекечинський Дмитро Олександрович

Київ 2026

Лабораторна робота №2. Розробка графічного редактора об’єктів

Мета: отримати вміння та навички використовувати інкапсуляцію, абстракцію типів, успадкування та поліморфізм на основі класів, запрограмувавши простий графічний редактор в об’єктно-орієнтованому стилі

Завдання:

1. Створити у середовищі розробки новий проект з ім’ям Lab2.
2. Скомпілювати проект і отримати виконуваний файл програми.
3. Перевірити роботу програми. За потреби налагодити програму.
4. Проаналізувати та прокоментувати результати та вихідний текст програми.
5. Оформити звіт.

Масив об'єктів: статичний масив, оскільки 7 mod 3 = 1. Розмір N = 7 + 100 = 107 об'єктів

"Гумовий" слід: пунктирна лінія чорного кольору, оскільки 7 mod 4 = 3.

Увід прямокутника: від центру до одного з кутів, оскільки 7 mod 2 = 1.

Відображення прямокутника: чорний контур зі світло-зеленим заповненням, оскільки 7 mod 5 = 2 та 7 mod 6 = 1.

Увід еліпсу: по двом протилежним кутам охоплюючого прямокутника, оскільки 7 mod 2 = 1.

Відображення еліпсу: чорний контур еліпсу без заповнення, оскільки 7 mod 5 = 2.

Позначка поточного типу об'єкту: в заголовку вікна, оскільки 7 mod 2 = 1.

Коди

Lab2.py

```
import tkinter as tk
from shapes import PointShape, LineShape, RectShape, EllipseShape

class EditorApp:
    def __init__(self, root):
        self.root = root
        self.current_tool = "Прямокутник"
        self.update_title()

        self.shapes = []
        self.max_shapes = 107
```

```
        self.start_x = None
        self.start_y = None
        self.rubber_id = None

        self.setup_menu()

        self.canvas = tk.Canvas(root, bg="white")
        self.canvas.pack(fill=tk.BOTH, expand=True)

        self.canvas.bind("<ButtonPress-1>", self.on_press)
        self.canvas.bind("<B1-Motion>", self.on_drag)
        self.canvas.bind("<ButtonRelease-1>", self.on_release)

    def update_title(self):
        self.root.title(f"Лабораторна робота №2 - {self.current_tool}")

    def set_tool(self, tool_name):
        self.current_tool = tool_name
        self.update_title()

    def setup_menu(self):
        menubar = tk.Menu(self.root)
        self.root.config(menu=menubar)

        file_menu = tk.Menu(menubar, tearoff=0)
        menubar.add_cascade(label="Файл", menu=file_menu)

        obj_menu = tk.Menu(menubar, tearoff=0)
        obj_menu.add_command(label="Крапка", command=lambda: self.set_tool("Крапка"))
        obj_menu.add_command(label="Лінія", command=lambda: self.set_tool("Лінія"))
        obj_menu.add_command(label="Прямокутник", command=lambda:
self.set_tool("Прямокутник"))
        obj_menu.add_command(label="Еліпс", command=lambda: self.set_tool("Еліпс"))
        menubar.add_cascade(label="Об'єкти", menu=obj_menu)

        help_menu = tk.Menu(menubar, tearoff=0)
        menubar.add_cascade(label="Довідка", menu=help_menu)

    def redraw_all(self):
        self.canvas.delete("all")
        for shape in self.shapes:
            shape.draw(self.canvas)

    def on_press(self, event):
        if len(self.shapes) >= self.max_shapes:
```

```
            return
        self.start_x = event.x
        self.start_y = event.y

        if self.current_tool == "Крапка":
            shape = PointShape(event.x, event.y, event.x, event.y)
            self.shapes.append(shape)
            shape.draw(self.canvas)

    def on_drag(self, event):
        if self.current_tool == "Крапка":
            if len(self.shapes) < self.max_shapes:
                shape = PointShape(event.x, event.y, event.x, event.y)
                self.shapes.append(shape)
                shape.draw(self.canvas)
            return

        if self.start_x is None:
            return

        self.redraw_all()

        if self.current_tool == "Лінія":
            self.rubber_id = self.canvas.create_line(
                self.start_x, self.start_y, event.x, event.y, dash=(4, 4), fill="black")
        elif self.current_tool == "Прямокутник":
            dx = abs(event.x - self.start_x)
            dy = abs(event.y - self.start_y)
            self.rubber_id = self.canvas.create_rectangle(
                self.start_x - dx, self.start_y - dy, self.start_x + dx, self.start_y + dy,
                dash=(4, 4), outline="black")
        elif self.current_tool == "Еліпс":
            self.rubber_id = self.canvas.create_oval(
                self.start_x, self.start_y, event.x, event.y,
                dash=(4, 4), outline="black")

    def on_release(self, event):
        if self.current_tool == "Крапка":
            self.start_x = None
            self.start_y = None
            return

        if self.start_x is None:
            return

        shape = None
```

```
        if self.current_tool == "Лінія":
            shape = LineShape(self.start_x, self.start_y, event.x, event.y)
        elif self.current_tool == "Прямокутник":
            shape = RectShape(self.start_x, self.start_y, event.x, event.y)
        elif self.current_tool == "Еліпс":
            shape = EllipseShape(self.start_x, self.start_y, event.x, event.y)

        if shape and len(self.shapes) < self.max_shapes:
            self.shapes.append(shape)

        self.redraw_all()

        self.start_x = None
        self.start_y = None

if __name__ == "__main__":
    main_window = tk.Tk()
    main_window.geometry("800x600")
    app = EditorApp(main_window)
    main_window.mainloop()
```

shapes.py

```
class Shape:
    def __init__(self, x1, y1, x2, y2):
        self.x1 = x1
        self.y1 = y1
        self.x2 = x2
        self.y2 = y2

    def draw(self, canvas):
        pass

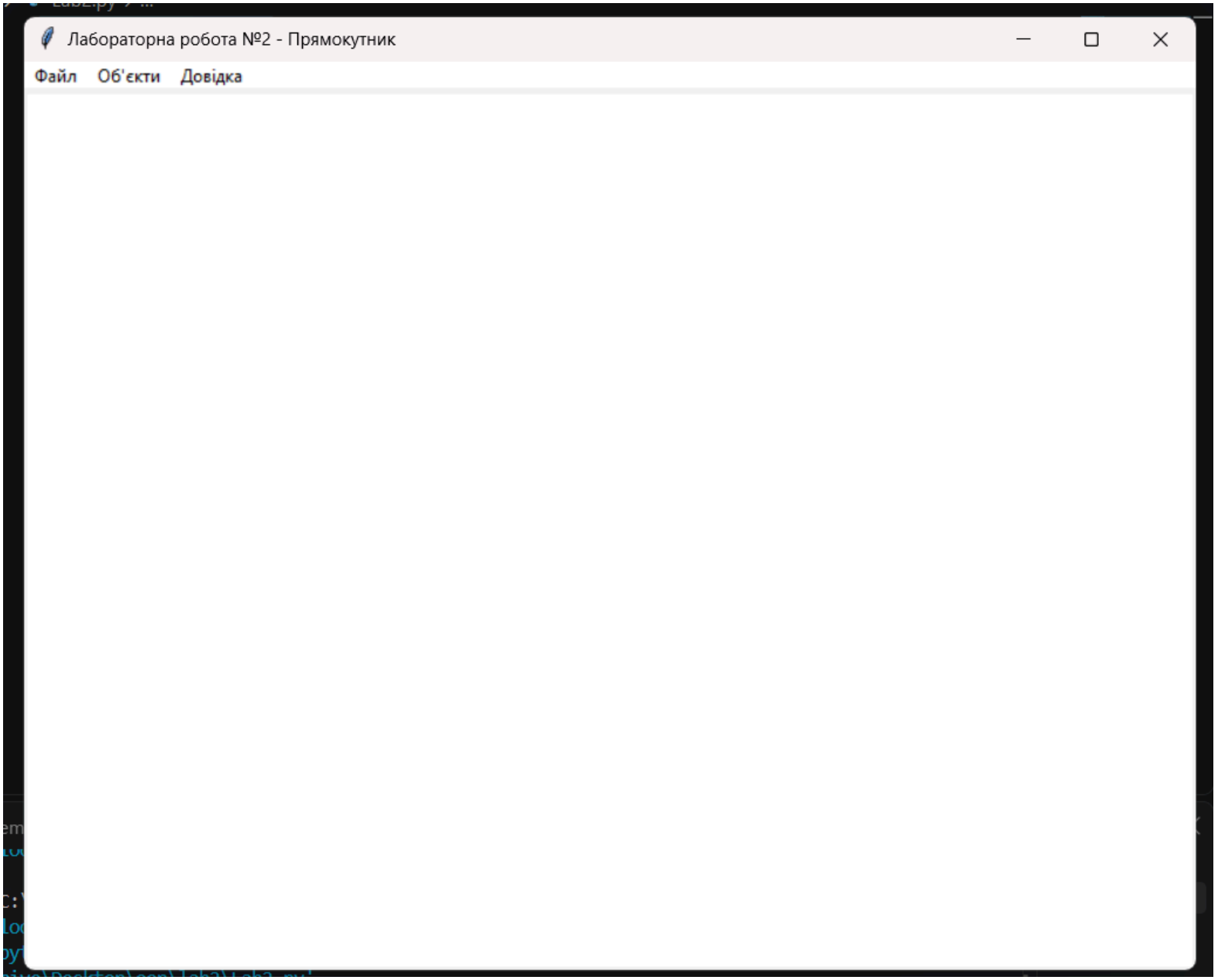
class PointShape(Shape):
    def draw(self, canvas):
        r = 4
        canvas.create_oval(
            self.x2 - r, self.y2 - r,
            self.x2 + r, self.y2 + r,
            fill="black", outline="black"
        )
```

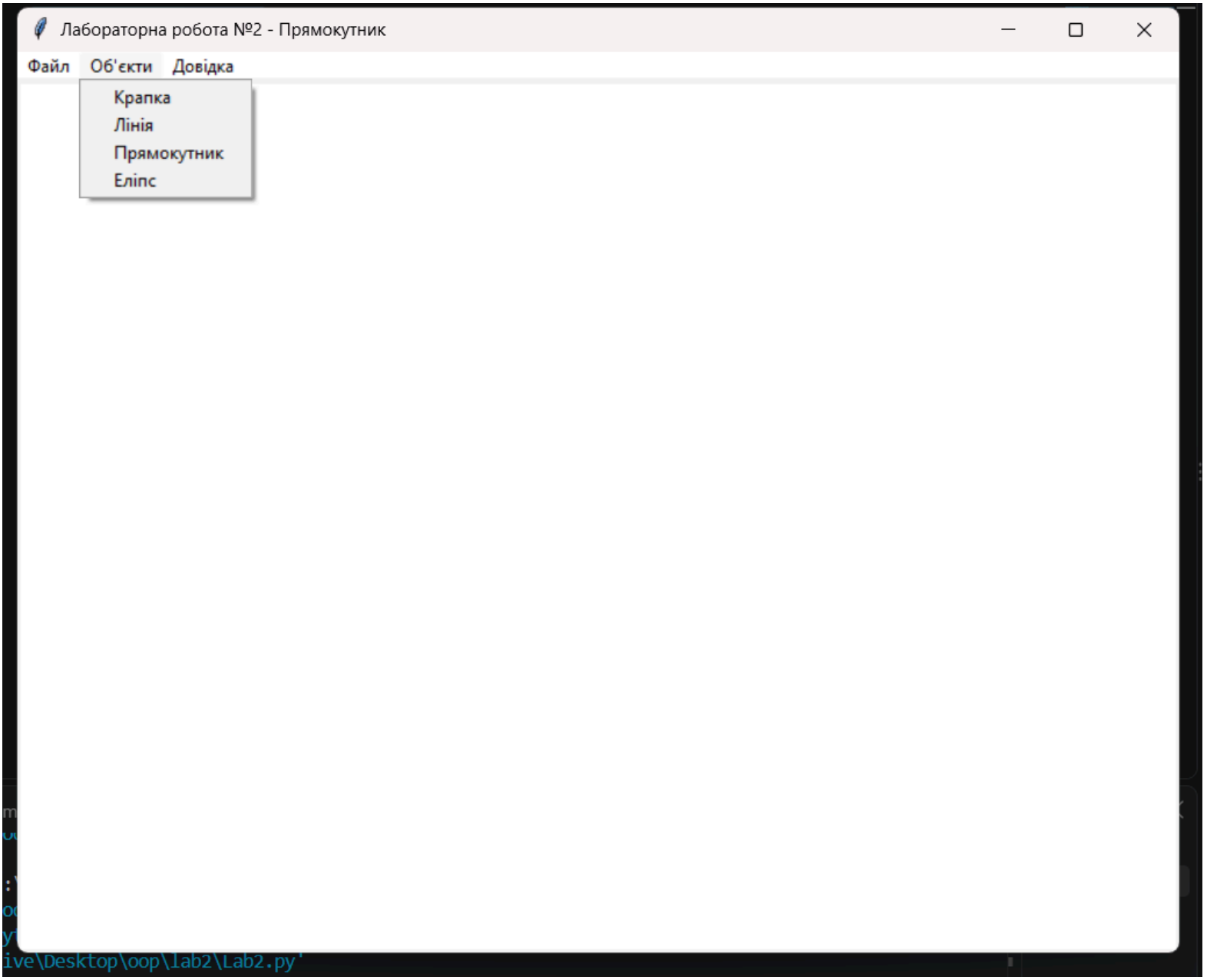
```
class LineShape(Shape):
    def draw(self, canvas):
        canvas.create_line(self.x1, self.y1, self.x2, self.y2, fill="black")

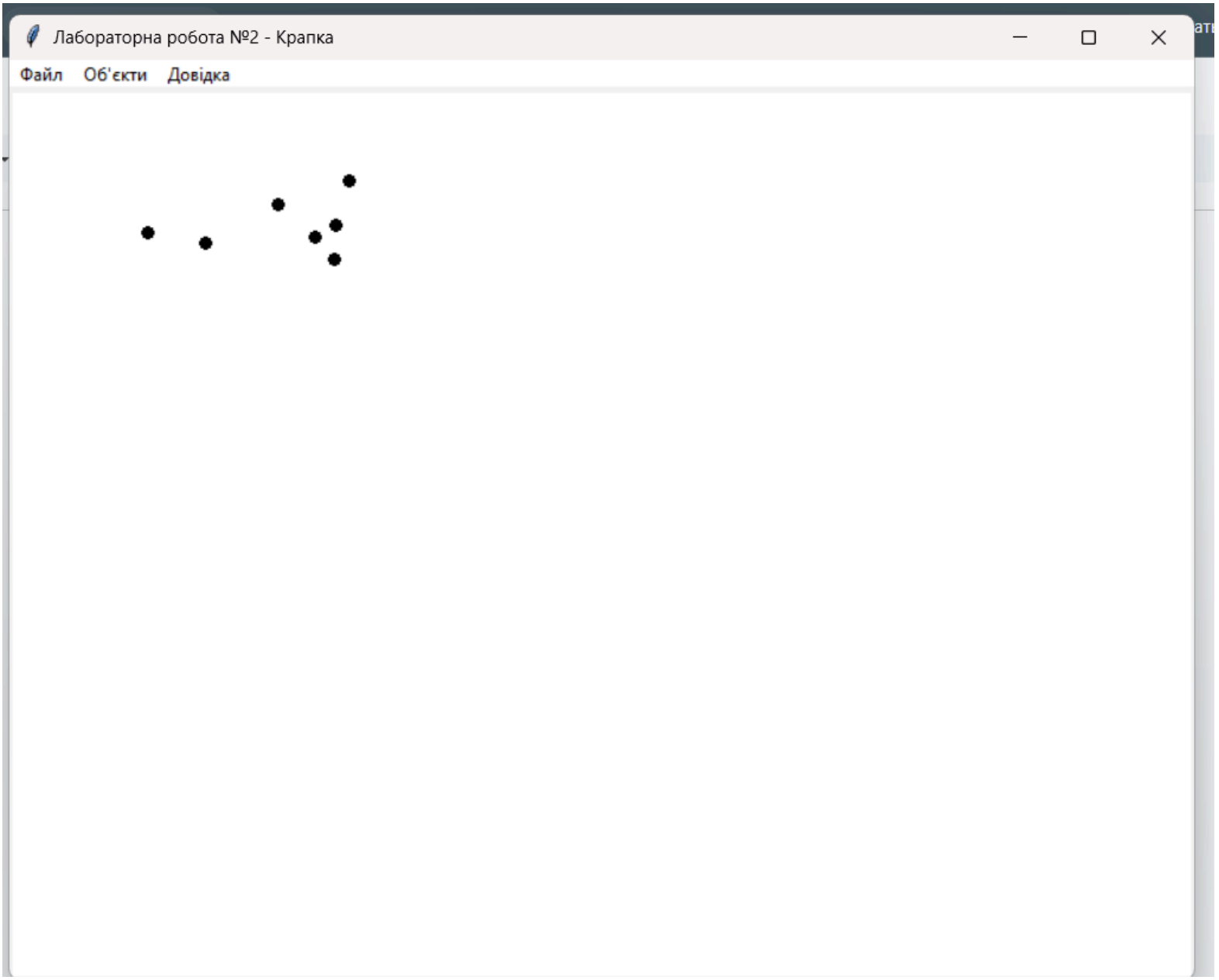
class RectShape(Shape):
    def draw(self, canvas):
        dx = abs(self.x2 - self.x1)
        dy = abs(self.y2 - self.y1)
        canvas.create_rectangle(
            self.x1 - dx, self.y1 - dy,
            self.x1 + dx, self.y1 + dy,
            outline="black", fill="lightgreen"
        )

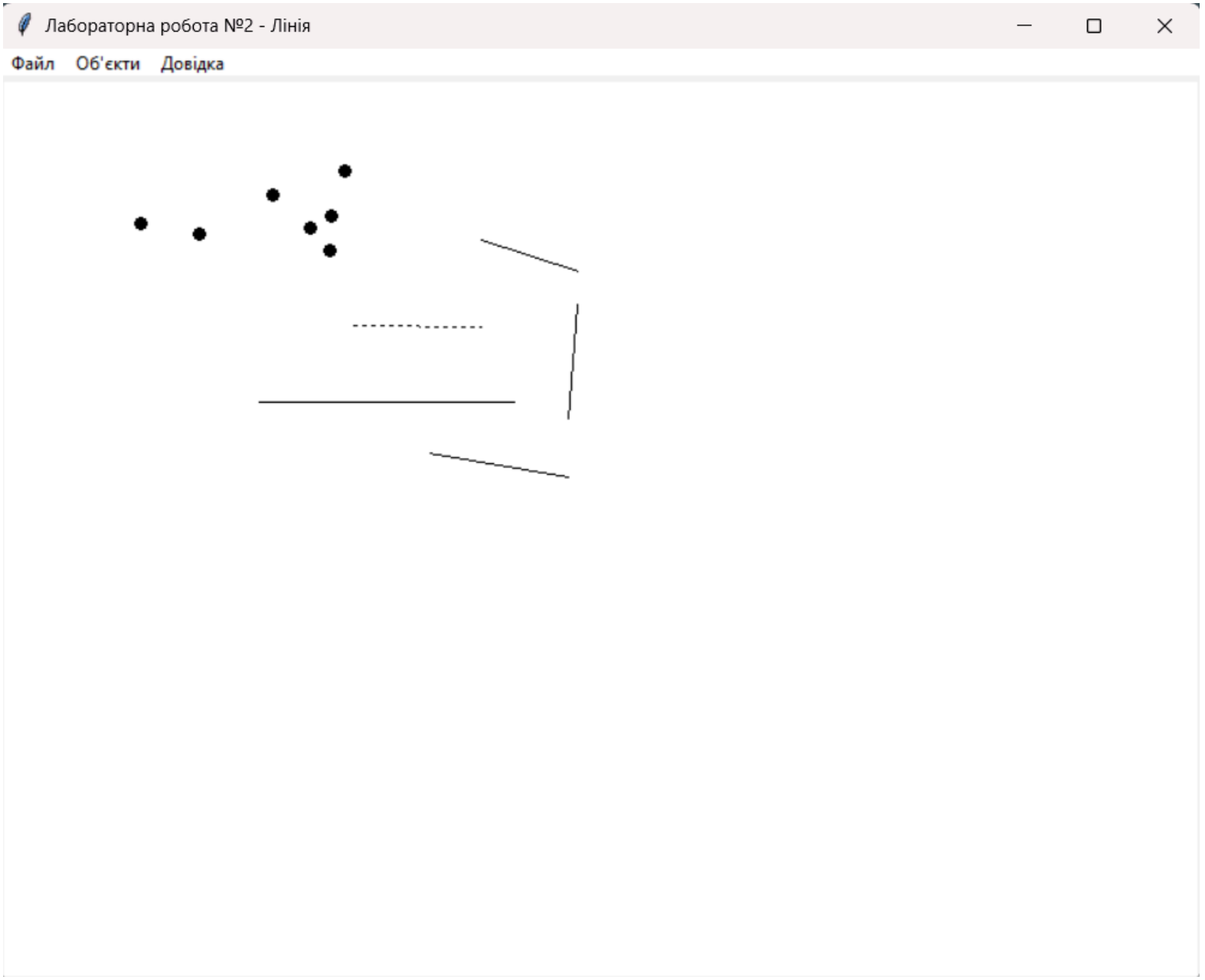
class EllipseShape(Shape):
    def draw(self, canvas):
        canvas.create_oval(
            self.x1, self.y1, self.x2, self.y2,
            outline="black", fill=""
        )
```

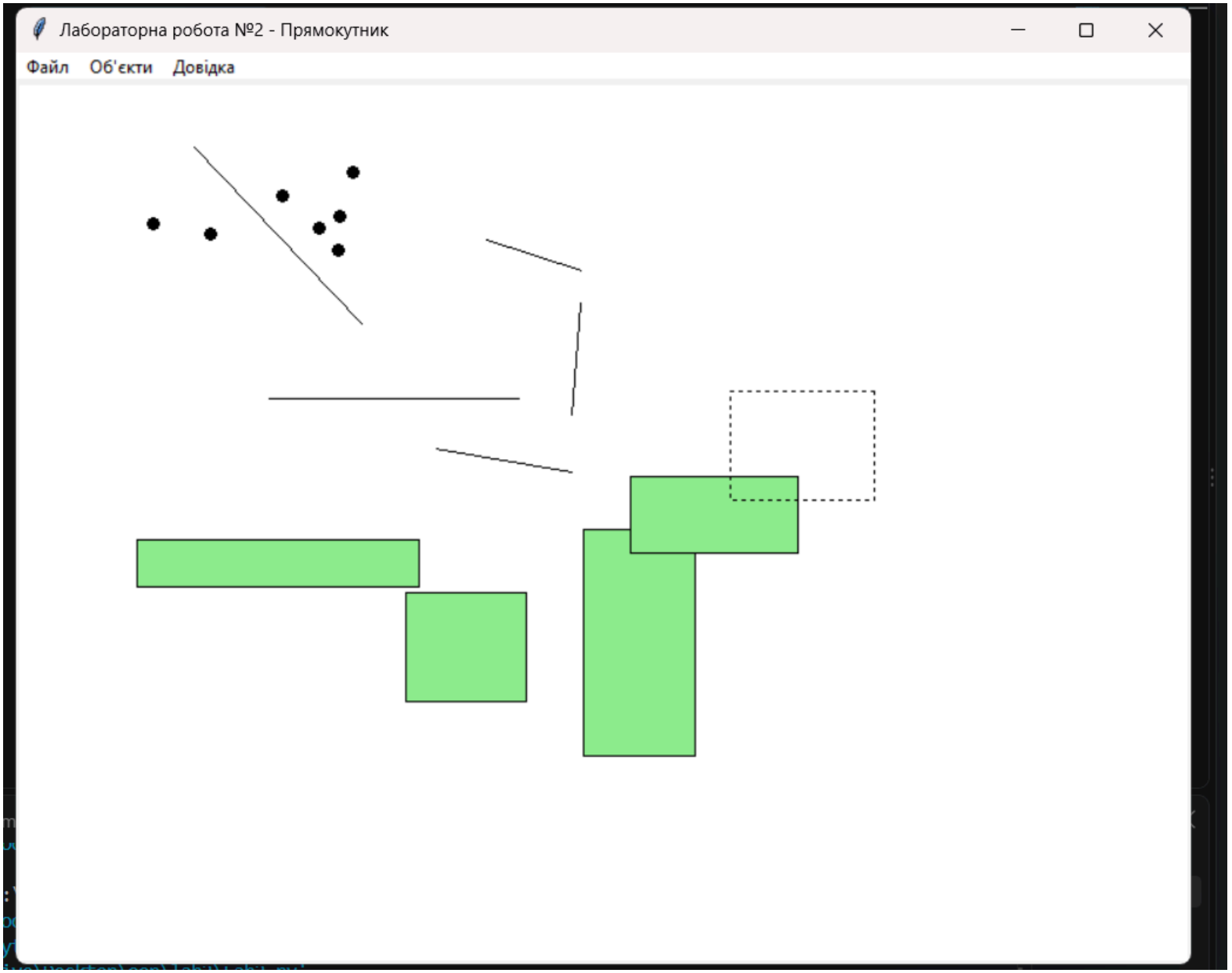
Виконання роботи

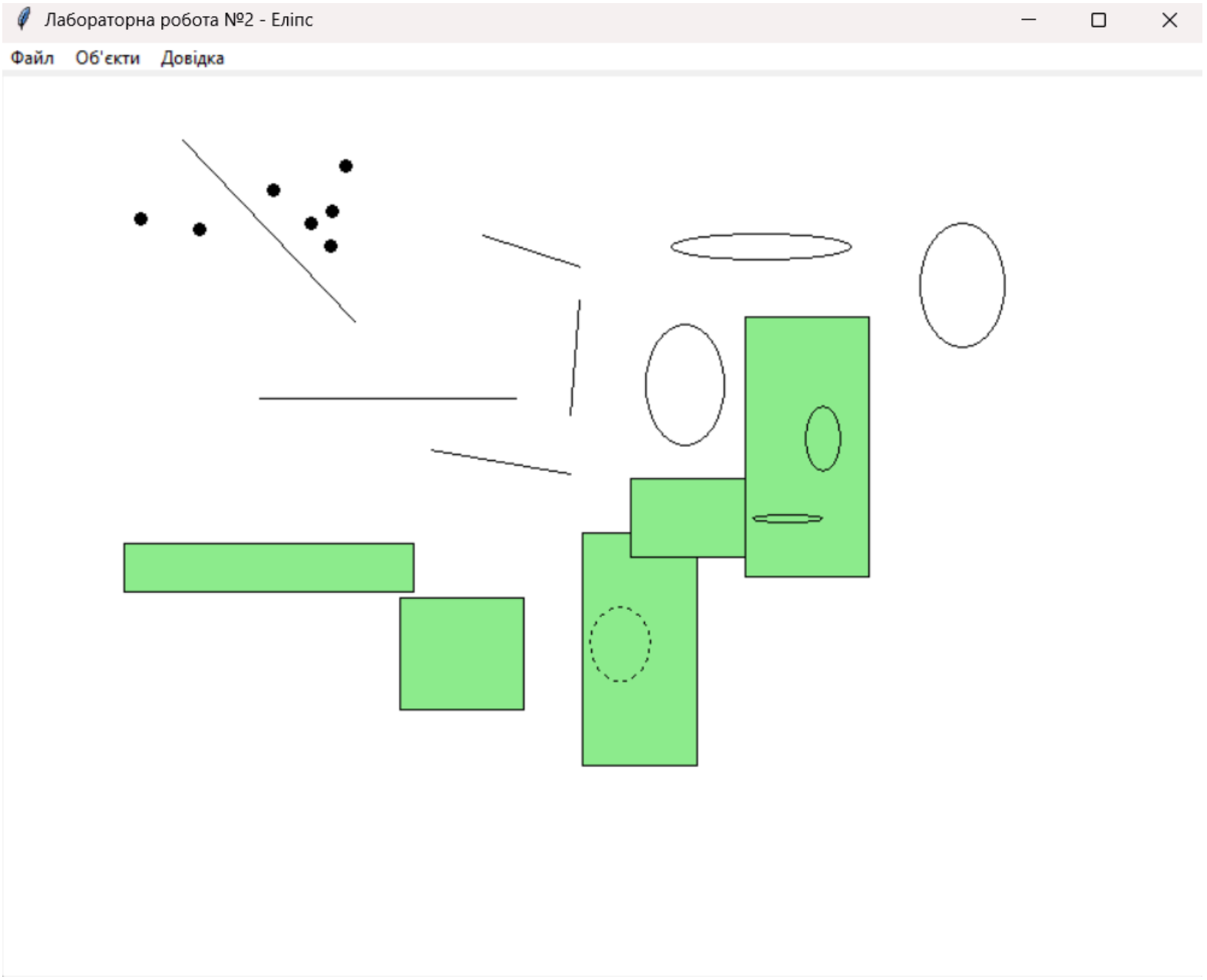

Лабораторна робота №2 - Прямокутник
Файл
Об'єкти
Довідка


Лабораторна робота №2 - Прямокутник
Файл
Об'єкти
Довідка
Крапка
Лінія
Прямокутник
Еліпс
ive\Desktop\oop\lab2\Lab2.py"

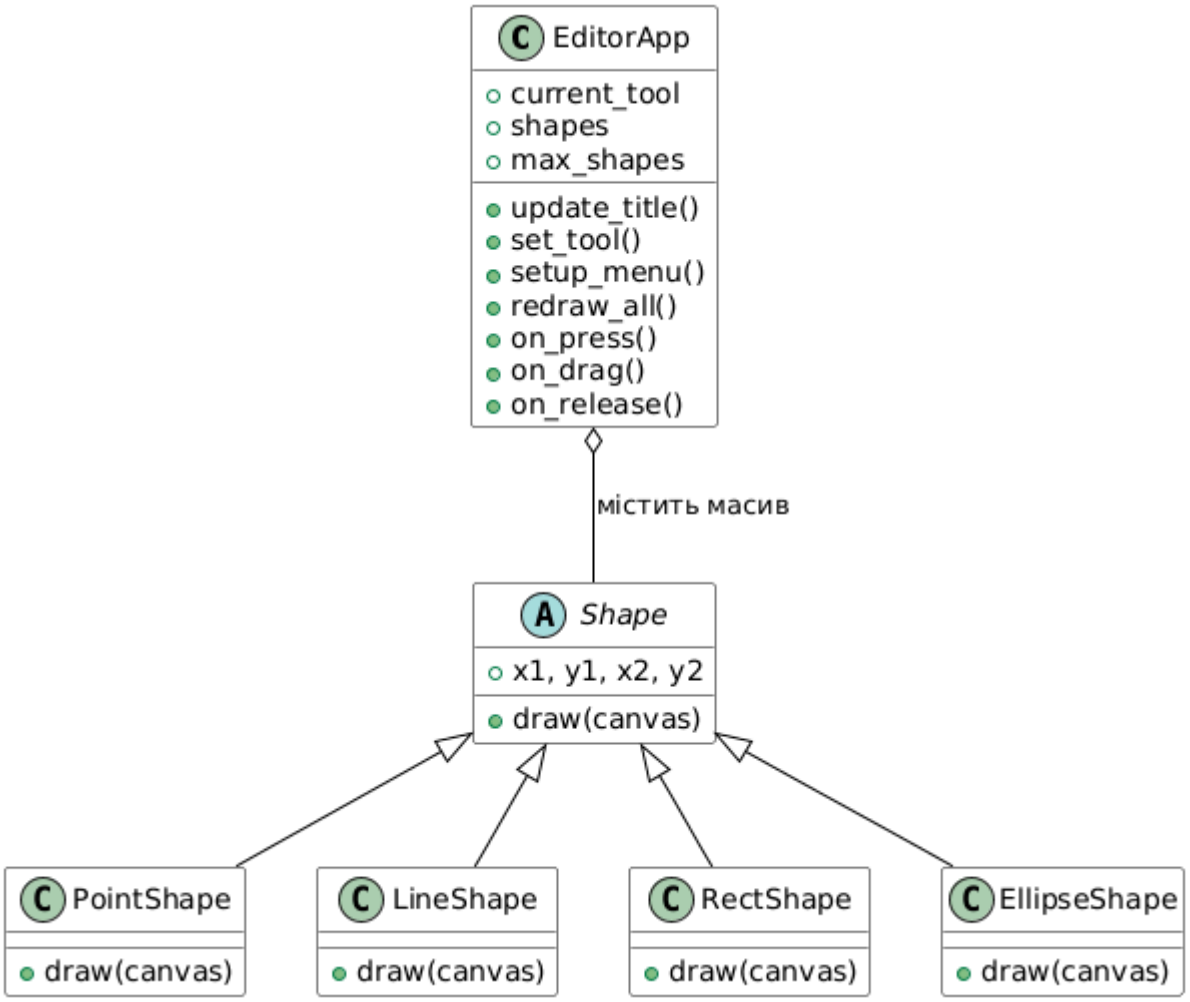

Лабораторна робота №2 - Крапка
Файл
Об'єкти
Довідка


Лабораторна робота №2 - Лінія
Файл
Об'єкти
Довідка


Лабораторна робота №2 - Прямокутник
Файл
Об'єкти
Довідка



Схема



Висновок

У цій лабораторній роботі я виконала завдання згідно зі своїм варіантом. Мною було розроблено програму, яка є простим графічним редактором, реалізованим в об’єктно-орієнтованому стилі. Під час виконання роботи я практично закріпила та навчилася використовувати такі принципи ООП, як інкапсуляція, абстракція типів, успадкування та поліморфізм